



UNIVERSITY OF PISA
MASTER'S DEGREE IN COMPUTER ENGINEERING

IMPLEMENTATION OF A CNN LAYER
COURSE ELECTRONICS & COMMUNICATION SYSTEMS

Author(s)
Francesco Rossi Paccani
Samuel Scarabelli

Academic year 2025/2026

Contents

1	Project Specifications	1
2	High-level Model [OPTIONAL]	3
3	RTL Design	5
3.1	Original Version	5
3.2	Optimized Version	6
4	Interface Specifications and Expected Behavior	8
5	Functional Verification	10
6	FPGA Implementation Results	12
6.1	CN synthesis	12
6.2	Optimized CN synthesis	13
7	Conclusions	16

CHAPTER 1

Project Specifications

The project relies on the implementation of a two-dimensional Convolutional Network (CN). A Convolutional Network permorms a convolution operation between a matrix and a filter. Let C_{in} be the input matrix, f be the filter and C_{out} be the output matrix, the output of a CN is defined by the following equation:

$$C_{out}^{(i)}(x, y) = \sum_{j=0}^{w-1} \sum_{k=0}^{h-1} f_i(j, k) C_{in}(x_i + j, y_i + k) \quad (1.1)$$

However, it is easier to understand the output of the network graphically. A visual example is shown in Fig. 1.1.

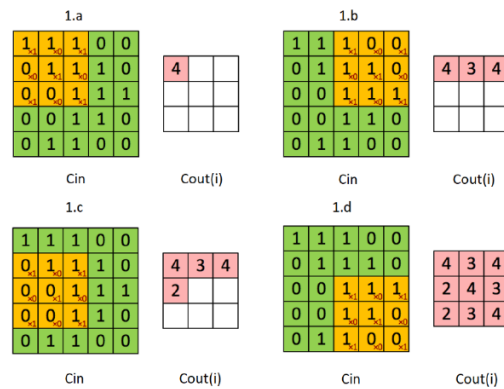


Figure 1.1: Graphical representation of the convolution output

The CN is a RSS (Synchronous Sequential Circuit), we can see the schema from the outside in Fig. 1.2.



Figure 1.2: *CN schema*

Both image and filter loading must be provided using x input signal which is 8-bit wide. i/f is used in order to distinguish matrix inputs from filter inputs. Inputs are considered valid and evaluated only when the input x_i signal is 1, otherwise the circuit state is retained. The correct procedure to use the circuit is to load the filter first ($i/f = 1$) and the matrix next ($i/f = 0$). Output values is provided with y output signal and must be considered valid only for one clock cycle ($y_valid = 1$). Input pixels can be provided in any order you consider convenient.

The newtowk must be resistant to unexpected changes of i/f signal and partial loading of filter elements.

CHAPTER 2

High-level Model [OPTIONAL]

We used C++ to build a high-level model of the convolution network. The model takes as input N , M , the matrix, and the filter; it evaluates the convolution and prints it (both in decimal and binary format).

Inside the VHDL testbench, we use the `std.textio` and `ieee.std_logic_textio` libraries to print the output via the ModelSim terminal. From there, we copy the output string and pass it to the C++ model, which compares the outputs to verify the correct behavior of the simulation.

```

INSERT MATRIX DIMENSION (N):
5
INSERT FILTER DIMENSION (M):
3
-----
INSERT MATRIX (length 25):
111000111100011110011001100
INSERT FILTER (length 9):
101010101
-----
CONVOLUTION OUTPUT (3x3)
4      3      4
2      4      3
2      3      4
-----
CONVOLUTION OUTPUT (BINARY FORMAT)
010000110100001001000011001000110100
-----
INSERT MODELSIM OUTPUT:
010000110100001001000011001000110100
-----
OUTPUT: OK

```

(a) *Output ok*

```

INSERT MATRIX DIMENSION (N):
5
INSERT FILTER DIMENSION (M):
3
-----
INSERT MATRIX (length 25):
111000111100011110011001100
INSERT FILTER (length 9):
101010101
-----
CONVOLUTION OUTPUT (3x3)
4      3      4
2      4      3
2      3      4
-----
CONVOLUTION OUTPUT (BINARY FORMAT)
010000110100001001000011001000110100
-----
INSERT MODELSIM OUTPUT:
010000110100001001000011001100110100
-----
OUTPUT: WRONG

```

(b) *Output wrong*

Figure 2.1: *C++ model output*

CHAPTER 3

RTL Design

Since the CN is a complex network, we adopted a bottom-up strategy during the design phase, dividing the entire network into smaller sub-networks and assembling them together. This made the designed VDH code simpler, more compact and more readable. We created two version of the Convolutional Network: the original one and its optimized version.

3.1 Original Version

The original CN consists of two distinct logic blocks: `Conv_2d_input`, used for storing external data, and `Conv_2d_output`, used to perform the convolution operation and a pulse generator to generate the `y_valid` pulse shape.

`Conv_2d_input` stores data within a component `Memory`. The `Memory` model is structured as a matrix of 1-bit D-Flip-Flops. Each matrix row consists of 8 DFFs, designed to store the 8-bit chunks in which data is periodically transmitted. Its input are the data, an enabler and a selector. The selector is used to enable the correct DDFS of the matrix to store the input data. `Conv_2d_input` is also composed of an asynchronous counter to track the number of valid data chunks stored in memory; this counter is also used to select the correct memory location for each 8-bit filter chunk. A ripple carry adder is used to obtain the value of the matrix memory selector from the counter output, ensuring that each 8-bit matrix chunk is stored in the correct memory address. Combinational logic is used to determine when the input data can be stored or not, in order to correctly save data under race conditions too. When all the data expected are stored correctly, an handshake signal (`en_execution`) is raised to communicate to the elaboration network to start to compute the stored data.

`Conv_2d_output` is the elaboration module of the CN that evaluates a new output element every clock cycle. It employs an Asynchronous Counter to track the number of evaluated output elements. To compute a single element, a chain of parallel multiplexers selects the specific matrix bits involved in the current evaluation, based on the counter's output. Subsequently, a chain of AND gates performs the bit by bit product between the selected matrix elements and the filter. A chain of Ripple Carry Adders then calculates the sum of the resulting product outputs. The final element value is stored inside a dedicated N-bit D-Flip-Flop, whose outputs contain the result of the operation (y). When the entire output elaboration is done, the logic block raises a flag (`convolution_ready`) to communicate the end of the elaboration. This signal used by a pulse generator, in order to generate the `y_valid` pulse.

`Conv_2d_output` is in a reset state until `en_execution` is zero and `Conv_2d_input` is resetted after the pulse of `y_valid`, in order to use the network with serial inputs.

3.2 Optimized Version

`Conv_2d_output_opt` is designed to reduce the critical path length caused by the Ripple Carry Adder chain and the multiplexers, leading into an increased maximum clock frequency. In this sub-module, the Ripple Carry Adder chain is replaced by a specialized synchronous counter, called `COUNTER_OPT`; this component takes the AND chain outputs as input and counts the number of set bits.

During the `COUNTER_OPT` operation, the main Asynchronous Counter stops counting and is enabled only once the `COUNTER_OPT` has produced a new output (one element of the final matrix). This mechanism was introduced because, while the original version produced one new output element every clock cycle (combinational logic), the optimized version produces a new output element every $M \times M$ clock cycles (synchronous sequential logic). As shown in the following chapters, this optimization will have a significant impact on network performance.

The multiplexers are replaced by a shift register loaded with the input matrix, where each 1-bit register stores 1 bit of the matrix. Instead of dynamically evaluating which matrix bit connects to the AND chain, the inputs of the AND ports are permanently connected to the specific registers of the shift register. The matrix bits are periodically shifted; since some shifts lead to an invalid state (meaning the AND chain is not wired to the correct input sub-matrix), these states are skipped by shifting the register values until a valid state is reached. This results in a shorter critical path, though it requires a larger number of clock cycles to produce the output.

To synchronize the shift register shifts and the `COUNTER_OPT` processing, a Finite State Machine (FSM) is used with the following states:

INACTIVE The initial state of the machine; the CN has not yet evaluated any output elements.

In this state, the shift register is loaded with the input matrix values before starting the processing of the first output element (transitioning to the `WAIT_ELABORATION` state).

WAIT_ELABORATION During this state, the `COUNTER_OPT` evaluates its output while the shift register remains disabled. When processing is complete, the `shift_counter` value is set (1 if the next shift leads to a valid configuration, M otherwise), and the FSM transitions to the `SHIFTING` state.

SHIFTING In this state, the shift register is enabled and performs 1 or M shifts, depending on the `shift_counter` value, before moving to the `WAIT_OUTPUT` state.

WAIT_OUTPUT In this state, `COUNTER_OPT` is loaded, and the system awaits its processing in the `WAIT_ELABORATION` state.

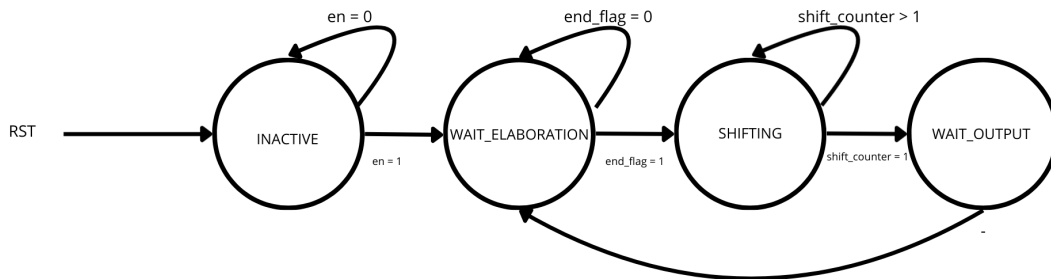


Figure 3.1: *FSM state diagram*

CHAPTER 4

Interface Specifications and Expected Behavior

In order to correctly use this Convolution Network Model we recommend to first keep the `reset` input set at least for 1 clock cycle. After this you have to reset the `i_f` input and load one filter data chunk into the `x` input; after this set the `x_valid` input for at least one clock cycle. Repeat this operation for every filter data chunk; provide the data chunks following the example shown in Fig. 4.1. Given the project specifications, it is impossible to save the matrix before the filter. To load matrix data perform the same procedure used with the filter, only set the `i_f` input. The output will be transmitted over the `y` signal and will be available only during the time the `y_valid` signal will be set (only for 1 clock cycle). When the `y_valid` signal resets the network will be ready to store a new filter and matrix in order to perform a new evaluation.

During the data input phase only inputs that will satisfy the previous directions will be counted as valid; otherwise the network will hung waiting for a correct input piloting. During the evaluating phase every input will be ignored otherwise they can change the data store in the memory interfering with the calculations.

FIRST 8 BIT CHUNK

1 ^[24]	0 ^[23]	1 ^[22]	0 ^[21]	0 ^[20]
0 ^[19]	1 ^[18]	0 ^[17]	1 ^[16]	0 ^[15]
0 ^[14]	0 ^[13]	1 ^[12]	1 ^[11]	1 ^[10]
0 ^[9]	1 ^[8]	0 ^[7]	1 ^[6]	0 ^[5]
0 ^[4]	1 ^[3]	1 ^[2]	0 ^[1]	0 ^[0]

0 ^[7]	x[7]
1 ^[6]	x[6]
0 ^[5]	x[5]
0 ^[4]	x[4]
1 ^[3]	x[3]
1 ^[2]	x[2]
0 ^[1]	x[1]
0 ^[0]	x[0]

SECOND 8 BIT CHUNK

1 ^[24]	0 ^[23]	1 ^[22]	0 ^[21]	0 ^[20]
0 ^[19]	1 ^[18]	0 ^[17]	1 ^[16]	0 ^[15]
0 ^[14]	0 ^[13]	1 ^[12]	1 ^[11]	1 ^[10]
0 ^[9]	1 ^[8]	0 ^[7]	1 ^[6]	0 ^[5]
0 ^[4]	1 ^[3]	1 ^[2]	0 ^[1]	0 ^[0]

0 ^[15]	x[7]
0 ^[14]	x[6]
0 ^[13]	x[5]
1 ^[12]	x[4]
1 ^[11]	x[3]
1 ^[10]	x[2]
0 ^[9]	x[1]
1 ^[8]	x[0]

(a) First chunk input

(b) Second chunk input

Figure 4.1: Example of first two chunk inputs. Iterate the method for more chunks.

CHAPTER 5

Functional Verification

The testing strategy employed two different testbench modules, each targeting different matrix and filter dimensions. We performed seven distinct tests to ensure the module's correct behavior under both normal operation and critical scenarios. The test descriptions are as follows:

Test 1 Correct driving of the module.

Test 2 Correct driving of the module, but with the extra bits of the last chunk set to 1 for both filter and matrix during the input phase.

Test 3 Correct driving of the module where both matrix and filter consist entirely of bits set to 1.

Test 4 Correct driving of the module where both matrix and filter consist entirely of bits set to 0.

Test 5 Incorrect driving of the module.

During the filter input phase, we applied all possible invalid input combinations:

- `x_valid = 0, i/f = 0`
- `x_valid = 0, i/f = 1`
- `x_valid = 1, i/f = 0`

The same applies to the matrix input phase:

- `x_valid = 0, i/f = 1`
- `x_valid = 0, i/f = 0`
- `x_valid = 1, i/f = 1`

Test 6 Same as Test 5, but attempting to change the filter input value during the processing phase.

Test 7 Correct driving of the module for two consecutive convolutional products, without resetting the module between the processing phase of the first product and the input phase of the second.

All performed tests produced outputs verified using a C++ high-level model (see chapter 2), ensuring the robustness of the module against all tested invalid scenarios and corner cases.

To validate the VHDL code we also utilized a `Tb_large_data` with large N and M values. The purpose of this testbench is to verify that our code works correctly for any positive value of N and M , specifically very large ones. Since loading the filter and matrix manually would have required many lines of code, we used some known cases. The first one consists of a matrix and filter of all 1s. The second one consists of a filter of all 1s and a matrix of alternating 1s and 0s. The output is also validated with the high-level model tool.

To design the network, we used a bottom-up strategy. In this case, it was very convenient to adopt this strategy because we could validate both modules with a single testbench. In fact, since the output of the `Conv_2d_output` depends on the correct saving of the data by the `Conv_2d_input`, we first designed the input network, validated it using the testbench, and then designed the output network using the existing testbench.

Overall, the used testbenches achieved 100% code coverage.

CHAPTER 6

FPGA Implementation Results

We implemented both the CN and its optimized version CN_OPT targeting the xc7z010clg400-1 device.

6.1 CN synthesis

We started by creating the `Conv_2d_wrapper` to ensure that Vivado analyzes all the Register-Logic-Register paths. The wrapper is simple: it places registers before the inputs and after the outputs of the CN. After importing all project modules into Vivado, we verified the elaborated design to see how Vivado maps our network. Subsequently, we performed the synthesis in `Out of context` mode to avoid mapping the circuit's pins, as we do not intend to physically mount it on the board at this stage. The synthesis completed with no errors or warnings. We then added a 125 MHz clock and defined the `Clock constraints`, re-running the synthesis with these constraints applied. The timing analysis yields the results shown in Fig. 6.1.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.225 ns	Worst Hold Slack (WHS): 0.254 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 265	Total Number of Endpoints: 265	Total Number of Endpoints: 128

Figure 6.1: CN timing analysis

We observe that the WNS is 1.225 ns, barely greater than 0. In this case we can increase the clock frequency. This positive WNS value indicates that our network is robust to temperature and voltage variations.

The power analysis produced the data shown in Fig. 6.2.

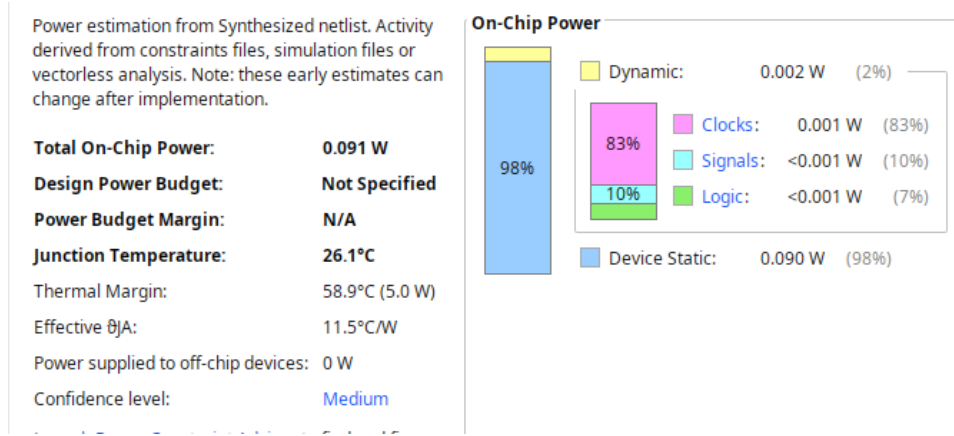


Figure 6.2: CN power analysis

As observed, the power consumption is dominated by the static power of the FPGA. The dynamic power associated with the design is approximately 2 mW. This dynamic component is mainly due to clock distribution, while the contribution of logic and signals is negligible, consistent with the small size of the design.

The area occupied by the circuit is shown in Fig. 6.3.

Name	Slice LUTs (17600)	Slice Registers (35200)
▼ N wrapper_conv_2d	110	128
reg_y_valid (D_Flip_Flop_parameterized1_1)	1	1
reg_y (D_Flip_Flop_parameterized3)	0	36
reg_x_valid (D_Flip_Flop_parameterized1_0)	0	1
reg_x (D_Flip_Flop)	0	8
reg_if (D_Flip_Flop_parameterized1)	0	1
▼ conv (Conv_2d)	109	81
> C_OUT (Conv_2d_output)	31	42
> C_INP (Conv_2d_input)	78	37

Figure 6.3: CN resource utilization

The network utilizes a total of 110 Slice LUTs and 128 Slice Registers, equivalent to approximately 0.6% and 0.4% of the logical resources available on the target platform, respectively.

6.2 Optimized CN synthesis

The same procedure was applied to the `Conv_2d_opt_wrapper`, which represents the wrapper for the optimized version. We performed the same analysis and obtained the data shown in the figures below.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.135 ns	Worst Hold Slack (WHS): 0.254 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 390	Total Number of Endpoints: 390	Total Number of Endpoints: 178

All user specified timing constraints are met.

Figure 6.4: Optimized CN timing analysis

Regarding the timing analysis (Fig. 6.4), we can see that the WNS is significantly greater than 0. In this scenario, we could achieve a faster network by increasing the clock frequency, as there is a substantial margin (maintaining $WNS > 0$). Alternatively, if we maintain the current frequency, the optimized version proves to be more efficient than the original one.

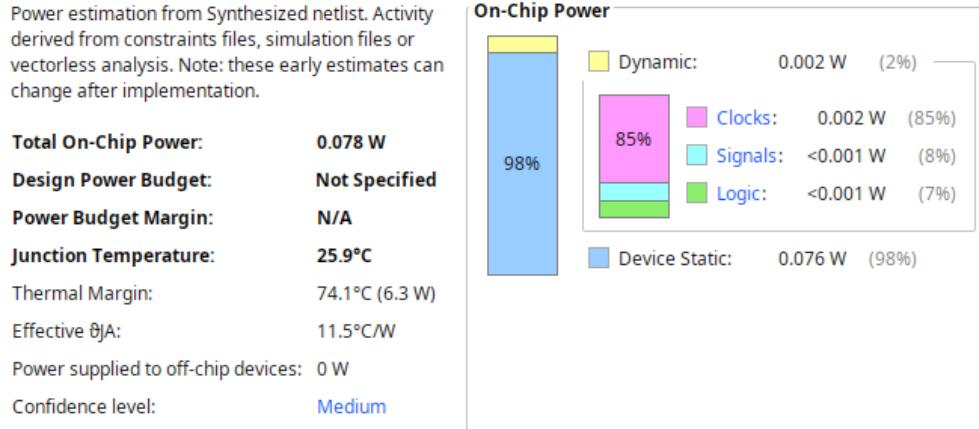


Figure 6.5: Optimized CN power analysis

As expected, the optimized version consumes less power than the original one; specifically, with $N = 5$, $M = 3$, we achieved a reduction of 13 mW, which is approximately 14%.

Name	Slice LUTs (17600)	Slice Registers (35200)
▼ N wrapper_conv_2d_opt	55	178
> I conv (Conv_2d_opt)	54	131
I reg_if (D_Flip_Flop_parameterized1)	0	1
I reg_x (D_Flip_Flop)	0	8
I reg_x_valid (D_Flip_Flop_parameterized1_0)	0	1
I reg_y (D_Flip_Flop_parameterized3)	0	36
I reg_y_valid (D_Flip_Flop_parameterized1_1)	1	1

Figure 6.6: Optimized CN resource utilization

In the case shown in Fig. 6.6, the network uses a total of 55 Slice LUTs and 178 Slice

Registers, equivalent to approximately 0.3% and 0.5% of the resources, respectively.

The optimized version is not only more efficient for $N = 5$ and $M = 3$, but its main advantage is better scalability as M and N increase. For instance, let us examine the timing report for both networks when $N = 27$ and $M = 15$.

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): -57.080 ns	Worst Hold Slack (WHS): 0.254 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): -444.523 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 9	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 8175	Total Number of Endpoints: 8175	Total Number of Endpoints: 3607
Timing constraints are not met.		

Figure 6.7: Standard CN timing analysis with $N=27$ and $M=16$

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 4.190 ns	Worst Hold Slack (WHS): 0.254 ns	Worst Pulse Width Slack (WPWS): 3.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 10937	Total Number of Endpoints: 10937	Total Number of Endpoints: 4621
All user specified timing constraints are met.		

Figure 6.8: Optimized CN timing analysis with $N=27$ and $M=16$

In Figs. 6.7 and 6.8, the difference between the two implementations is more evident. In this case, the original version does not meet the timing constraints due to the multiplexers and the Ripple Carry Adder chain. In contrast, the optimized version operates with a very high WNS even with larger parameters.

CHAPTER 7

Conclusions

Both the original and optimized versions of the component are ready to be deployed on a real FPGA. As demonstrated in Chapter 6, the optimized version is more robust, efficient, and scalable than the original one. The proposed designs are scalable by varying the parameters N and M . Overall, thanks to the bottom-up design strategy, the implementation of the network proved easier to understand and develop. As a result, the optimized version is reliable and capable of handling high-frequency signals.